

Standalone exact certificate for the universal MinVol contact bound

Repository source: [certificate/README.md](#)

Contents

Result represented by this package	1
Standalone architecture	1
Public documentation map	2
Quick replay	2
Exact proof path	2
Proof authority and audits	3
Adversarial rejection coverage	3
Construction-environment validation	3

Result represented by this package

For every three-dimensional convex body K of constant width d , the package proposes the universal inequality

$$\text{Vol}(K) \geq \frac{26220940089713\pi}{200000000000000} d^3 = 0.41187756378030227424817892598 \dots d^3.$$

Status: internally replayed proposed exact universal certificate. The package has passed its exact Python replay, an independent binary64 Python audit, and seven adversarial rejection tests in the construction environment. The archived base-R audit has independently passed on the user's machine. The result has not yet been externally reproduced, peer reviewed, or published.

This package does not prove sharpness, equality, rigidity, identification of a minimizer, or Meissner extremality.

Standalone architecture

The verifier imports no private repository package and does not use a sibling research directory. Its complete runtime proof closure is local to this directory:

- `certify_universal_bound.py` assembles and verifies the exact theorem;
- `exact_geometry.py` supplies deterministic integer and `Fraction` arithmetic;
- `certificates/reference_handoff_certificate.json` records the immutable predecessor rows through the hand-off and the exact reference-fan regression target;
- `certificates/reference_near_jung_fan.npz` is the immutable reference fan;
- `certificates/universal_contact_fan.npz` is the adjusted 98,306-ray fan;
- `certificates/refined_lower_cell_1.npz` through `refined_lower_cell_4.npz` are the exact replacement cells;
- `test_universal_bound.py` supplies reconstruction and regression tests;
- `test_adversarial_mutations.py` perturbs temporary copies of seven theorem-critical objects and requires rejection;
- the TSV and CSV files are byte-checked exact/audit renderings; and
- `certificates/universal_contact_bound_certificate.json` is the authoritative assembled machine-readable theorem certificate.

The reference handoff JSON is deliberately narrow. It is a trusted immutable proof object for predecessor rows and reference-fan regression values; it is not presented as a second standalone theorem certificate. The final verifier replays the four replacement rows, every final fan-ray margin, the fan volume, section hulls, determinant transfer, all three cap allocations, cross-axis disjointness, and the universal handoff.

No private Git history, branch name, commit identifier, bundle, chat handoff, or private research object is included.

Public documentation map

The certificate is accompanied by:

- [CERTIFICATE_SPECIFICATION.md](#), defining the JSON, NPZ, TSV, and CSV interfaces;
- [TRUST_BOUNDARY.md](#), distinguishing exact authority from audits and trusted infrastructure;
- [CLAIMS_AND_LIMITATIONS.md](#), fixing the public claim boundary; and
- [INDEPENDENT_AUDIT.md](#), recording implementations, environments, mutations, and remaining external-review needs.

The proof overview, claim graph, exact constants, source ledger, and internal proof audit are under `../proof/`. The complete reviewer path is linked from the repository [README.md](#).

Quick replay

Create and activate a Python environment, install the pinned requirement from the repository root, and run:

```
./run_python_checks.sh
```

The expected final markers are:

```
MINVOL UNIVERSAL CONTACT-BOUND CERTIFICATE: EXACT
PROPOSED UNIVERSAL LOWER BOUND ABOVE 0.4118775: EXACTLY CERTIFIED BY THIS PACKAGE
MINVOL UNIVERSAL CONTACT-BOUND BINARY64 AUDIT: PASS
Ran 10 tests
OK
Ran 7 tests
OK
MINVOL ADVERSARIAL MUTATION SUITE: PASS
MINVOL EXACT-CONSTANTS APPENDIX: CURRENT
MINVOL RELEASE-DOCUMENT CHECK: PASS
MINVOL PAPER-CERTIFICATE RECONCILIATION: PASS
```

The committed transcript `transcripts/base_r_audit.txt` records a successful base-R replay on the user's machine. To reproduce or deliberately refresh it, run from the repository root:

```
./run_r_checks.sh 2>&1 | tee certificate/transcripts/base_r_audit.txt
```

```
python3 refresh_checksums.py
```

The required marker is:

```
MINVOL UNIVERSAL CONTACT-BOUND BASE-R AUDIT: PASS
```

After both language paths pass:

```
./run_all.sh
```

The final all-language marker is:

```
MINVOL UNIVERSAL CONTACT-BOUND REPLAY: PASS
```

Exact proof path

The exact verifier establishes a gap-free circumradius split at

$$R_* = \frac{305695601}{500000000} = 0.611391202.$$

For the lower branch, it retains the immutable predecessor rows, replaces the formerly insufficient 03A1 interval by four exact doubled-angular-grid cells, and confirms that 03B02 is the unique bottleneck.

For the near-Jung branch, it reconstructs the cubical spherical fan with 98,306 vertices, 196,608 triangular faces, 294,912 edges, and 24,833 stabilizer orbits. It then verifies:

- an exact minimum fan margin above $1e-12$;
- the determinant transfer $D(e) \geq 1 - S/2 - S^2/2$;
- dominant-edge coordinate-width bounds $H_{00} \leq 1 + S/2$ and $H_{11}, H_{22} \leq 1 + S/6$;
- a forced-core lower bound of $0.4117357103127771735489695\dots$;
- three optimized antipodal cap-pair bounds totaling $0.00014396090205525818382845\dots$;
- exact six-cap disjointness; and
- a near-Jung lower bound of $0.41187967121483243173279798\dots$.

The exact near-Jung surplus above the upper enclosure of the theorem value is $0.000002107434530157484619055\dots$

Proof authority and audits

The proof authority is:

1. the immutable NPZ/JSON proof objects;
2. `exact_geometry.py` and `certify_universal_bound.py`;
3. exact Python integer and `fractions.Fraction` arithmetic; and
4. byte-identical reconstruction of the assembled certificate and exact audit renderings.

`audit_universal_bound.py` is an independent binary64 reconstruction. It is an audit, not the proof authority. `audit_universal_bound.R` independently checks archived exact decimal renderings and reconstructs the cap allocations and assembly using base R; it is also an audit, not the proof authority.

Adversarial rejection coverage

The mutation suite never changes a tracked repository file. It creates a temporary copy, changes exactly one targeted object, and requires the production verifier path to reject it. The seven committed cases are:

1. a lower-row interval endpoint;
2. a final-fan radial numerator;
3. an inherited S-procedure multiplier;
4. a section-hull vertex;
5. a cap-allocation bracket endpoint;
6. a cross-axis disjointness threshold; and
7. one digit of the theorem coefficient.

The first three tests bypass only the immutable-file fingerprint and exercise the underlying exact semantic checks. The remaining four exercise the same byte-identical reconstruction gate used by the full exact verifier. The archived result is in `transcripts/adversarial_mutations.txt`.

NumPy is trusted only to read immutable integer arrays and to support the independent numerical audit. Floating-point arithmetic is not used to justify the theorem.

Construction-environment validation

Completed here:

- exact standalone certificate replay;
- byte-identical assembled-certificate reconstruction;
- reference-fan fingerprint and exact-margin regression;
- independent binary64 audit;
- ten Python unit tests, including explicit standalone-path checks;
- seven adversarial mutation tests covering every requested proof-object class;
- construction-time semantic comparison against the source theorem certificate; all ten checked theorem-critical fields match (see `transcripts/export_equivalence.txt`);
- pre- and post-replay SHA-256 verification;

- automatic reconciliation of the lean Paper v1 theorem constants and gap-free lower-branch table against the authoritative certificate JSON; and
- construction and visual/PDF preflight of the current manuscript and complete paired Markdown/PDF release-document set;
- generated exact-constants appendix verification; and
- release-document presence, claim-boundary, and relative-link verification.

Completed on the user's machine:

- a clean Python 3.14 virtual-environment replay;
- the independent base-R audit, including the required final pass marker;
- pre- and post-replay checksum verification; and
- creation of the fresh-history root commit from a clean working tree.

Not completed here:

- a second base-R execution in the construction container, because `Rscript` is unavailable there;
- external mathematical review;
- final public copy edit and author sign-off; and
- final authorship, affiliation, licensing, citation, repository, release, and archival metadata.

The latter items remain release gates.